

Circuit Analysis with Python

Tiburonboy

juan1543cabrillo@sudomail.com

August 26, 2026

Abstract

This paper presents a procedure to analyze electric circuits which may contain resistors, capacitors, inductors, Op Amps, dependent and independent sources using the Python programming language. The procedure presented in this paper will use Modified Nodal Analysis to generate the network equations. It is shown that the SymPy and NumPy libraries can be used to generate symbolic network equations from a circuit's netlist and solve those equations with almost no effort. The procedure is efficient and less error prone compared to manual calculations.

Keywords: Linear Circuit Analysis, Modified Nodal Analysis, Symbolic Analysis, Controlled Sources, Element Stamps

1. Introduction

The purpose of this paper is to describe a procedure implemented in the Python programming language that will automatically generate symbolic network equations from a circuit's netlist. The Python programming language and the associated libraries such as SymPy and NumPy make analyzing electric circuits almost effortless and the JupyterLab Notebooks described in this paper can be used as template for analyzing almost any linear electric circuit.

Circuit analysis is a foundational skill necessary for a broad range of professionals whose work involves the design and development of electronic systems. Electrical Engineers and Circuit Designers are the core audience, as they rely on techniques like Kirchhoff's Laws to create efficient circuits, calculate voltage drops and current flows, and simulate how an electric circuit will behave before it's ever built. Fields like Power Systems Engineering, Control Systems Engineering and even Physics require this knowledge to understand energy distribution, optimize system performance and analyze complex physical models where circuits are used as analogues.

The Modified Nodal Analysis (MNA) is an essential extension of standard Nodal Analysis, and its primary benefits revolve around its systematic and comprehensive nature, particularly for computer-aided analysis. The key advantage of MNA is its ability to accommodate many types of circuit elements in a single, unified matrix formulation. Unlike traditional Nodal Analysis, which struggles to directly model voltage sources and components whose current is not easily defined by node voltages (such as dependent voltage sources and inductors), MNA introduces the currents through these elements as additional unknown variables. This standardization makes the process of formulating the circuit equations highly algorithmic and straightforward to automate, forming the backbone of professional circuit simulators like SPICE. Consequently, MNA eliminates the need for manual, ad-hoc techniques like supernodes or source transformations, allowing for the rapid and reliable analysis of large, complex, and linear circuits.

In 1975, a paper titled, *The Modified Nodal Approach to Network Analysis*, was published by [1]. This was the original scholarly paper on the subject. The analysis method they presented allows for the ability to process voltage sources and current-dependent circuit elements in a simple and efficient manner. The paper describes the formulation of the matrices, the use of stamps and a pivot ordering

strategy. The authors compare their algorithm to the tableau method, a circuit analysis technique, which was an analysis technique being described in scholarly papers at the time.

The analysis of electric circuits, whether done by hand or with the help of Python and MNA, fundamentally relies on the concept of ideal components. These are theoretical abstractions that possess perfect behavior (e.g., a resistor has only resistance, an inductor has only inductance) and ignore the inevitable imperfections of real-world devices, such as parasitic capacitance, lead resistance, or temperature dependence.

Furthermore, almost all conventional circuit analysis methods, including Nodal and Mesh analysis, rely on the assumption of a lumped-element model, which assumes the circuit is small enough that electrical effects occur instantaneously throughout the circuit, neglecting the time delay associated with the propagation of electromagnetic fields. This allows us to treat the circuit's current and voltage as functions of time alone, rather than of both position and time (as required for wave propagation or transmission line effects). Finally, the systematic matrix methods discussed (MNA) are designed specifically for linear time-invariant (LTI) circuits, where the relationship between voltage and current is linear (e.g., Ohm's law holds true) and the component values or connections do not change over time.

Symbolic circuit analysis is a formal technique where the behavior or characteristic of a circuit (like voltage gain or impedance) is calculated with the circuit components and frequency (or time) represented by symbols instead of numerical values.

For small circuits, the analytical expression (e.g., a transfer function) explicitly shows how each component affects the circuit's performance metrics (gain, poles, zeros, etc.). This allows a designer to immediately see which elements are dominant and how to modify component values to meet specifications. A symbolic expression remains valid across a range of component values and operating conditions (as long as the underlying model is valid). By using symbolic analysis the network equations in symbolic form can be useful to document a design or to include in a technical paper.

2. Analysis Method

Modified Nodal Analysis (MNA) was used to obtain the network equations. The MNA procedure provides an algorithmic method for generating a system of independent equations for linear circuit analysis. Once the schematic is drawn and the net list is exported, the network equations and solutions for the node voltages are easily obtained as shown in this paper. A JupyterLab notebook [2] was written to perform the analysis and write the content of this paper. The code in the JupyterLab notebook can be used as a template to analyze almost any type of linear circuit.

The schematic for the example circuits below were drawn using LTSpice and the netlist was exported for the analysis. Drawing a circuit with a schematic capture program and exporting the netlist reduced the possibilities for errors as the analysis proceeds. The analysis begins with generating the network equations from the circuit's netlist by calling:

```
report, network_df, df2, A, X, Z = SymMNA.smna(filter_net_list)
```

The function `SymMNA.smna()` is described in [3]. The SymPy `solve` is capable of working through the algebra to solve the system of equations and can even find the roots of the numerator and denominator polynomials in some cases. Analytic expressions are easily generated for the circuit's node voltages.

Numerical values can be substituted for the component values and NumPy can be used to find numerical solutions for the network equations.

3. Analysis Results

Symbolic analysis presented in this paper employs the MNA technique to generate network equations from the circuit's netlist where the element values are represented by symbols. SymPy is used to solve the system of equations in symbolic form to obtain analytic expressions for the circuit's node voltages. The symbolic expressions can in some cases provide a deeper understanding of how each component contributes to the circuit's operation that can complement numerical simulations.

The following Python modules are used in the procedure.

```
from sympy import *
import numpy as np
from tabulate import tabulate
from scipy import signal
import matplotlib.pyplot as plt
import pandas as pd
import SymMNA
from IPython.display import display, Markdown, Math, Latex
init_printing()
```

The netlists for the example circuits are generated by LTSpice from the schematic. The nodes in the circuits were labeled in the schematic, otherwise LTSpice will use default labels such as N001, N002 etc. and the SymMNA.smna function requires integer values for the node numbers and these need to be consecutively ordered with no gaps in the numbering.

Generation of the netlist from a schematic capture program is convenient and less error prone than generating the netlist by hand. A visual inspection of the schematic ensures that the circuit to be analyzed is correct and it follows that the netlist is also correct. This is especially true for larger, more complicated schematics.

The procedure for analyzing electrical circuits with Python is best explained by considering the example circuits described below.

3.1. Example 1

The circuit shown in Figure 1 contains three resistors, one capacitor, one inductor and one independent voltage source. The nodes have been numbered and the reference node is indicated by the ground symbol. The schematic was drawn using LTSpice and the netlist was exported and copied into the code cell below.

The circuit is a typical of circuits found in circuit analysis text books. The independent voltage source spans nodes 1 and 2. Typically, these nodes would be considered a super-node when applying KCL to obtain a set of network equations. When using the MNA procedure, extra equations are generated to deal with voltage sources that span circuit nodes.

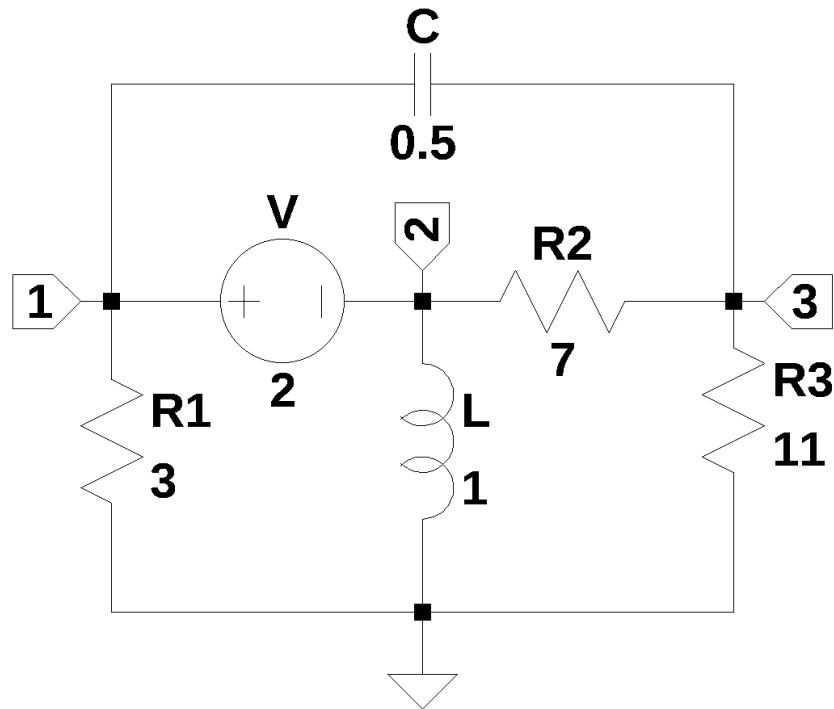


Figure 1: Schematic for example 1.

The code in the cell below formulates the network equations from the circuit's netlist using MNA.

```
net_list = '''
R1 1 0 3
R2 3 2 7
R3 3 0 11
C 3 1 0.5
L 2 0 1
V 1 2 2
'''

# generate the MNA matrices.
report, network_df, i_unk_df, A, X, Z = SymMNA.smna(net_list)

# Put matrices into SymPy
X = Matrix(X)
Z = Matrix(Z)

# assemble the system of equations
NE_sym = Eq(A*X, Z)

# turn the free symbols into SymPy variables.
var(str(NE_sym.free_symbols).replace('{', '').replace('}', ''))

# display the network equations
temp = ''
for i in range(len(X)):
    temp += '${:s}$<br>'.format(latex(Eq((A*X)[i:i+1][0], Z[i])))
```

Markdown(temp)

$$\begin{aligned}
0 &= -Cs v_3 + I_V + v_1 \left(Cs + \frac{1}{R_1} \right) \\
0 &= I_L - I_V + \frac{v_2}{R_2} - \frac{v_3}{R_2} \\
0 &= -Cs v_1 + v_3 \left(Cs + \frac{1}{R_3} + \frac{1}{R_2} \right) - \frac{v_2}{R_2} \\
V &= v_1 - v_2 \\
0 &= -I_L L s + v_2
\end{aligned} \tag{1}$$

Use the solve function to obtain the expressions for the node voltages and unknown currents. The node voltages in symbolic form are displayed below.

```
U_sym = solve(NE_sym,X)

temp = ''
for i in U_sym.keys():
    temp += '${:s} = {:s}$<br>'.format(latex(i), latex(U_sym[i]))

Markdown(temp)
```

$$\begin{aligned}
v_1 &= \frac{CR_1 R_2 R_3 V s + LR_1 V s + R_1 R_2 V + R_1 R_3 V}{CLR_1 R_2 s^2 + CLR_2 R_3 s^2 + CR_1 R_2 R_3 s + LR_1 s + LR_2 s + LR_3 s + R_1 R_2 + R_1 R_3} \\
v_2 &= \frac{-CLR_1 R_2 V s^2 - CLR_2 R_3 V s^2 - LR_2 V s - LR_3 V s}{CLR_1 R_2 s^2 + CLR_2 R_3 s^2 + CR_1 R_2 R_3 s + LR_1 s + LR_2 s + LR_3 s + R_1 R_2 + R_1 R_3} \\
v_3 &= \frac{CR_1 R_2 R_3 V s - LR_3 V s}{CLR_1 R_2 s^2 + CLR_2 R_3 s^2 + CR_1 R_2 R_3 s + LR_1 s + LR_2 s + LR_3 s + R_1 R_2 + R_1 R_3} \\
I_V &= \frac{-CLR_1 V s^2 - CLR_3 V s^2 - CR_1 R_2 V s - CR_1 R_3 V s - CR_2 R_3 V s - LV s - R_2 V - R_3 V}{CLR_1 R_2 s^2 + CLR_2 R_3 s^2 + CR_1 R_2 R_3 s + LR_1 s + LR_2 s + LR_3 s + R_1 R_2 + R_1 R_3} \\
I_L &= \frac{-CR_1 R_2 V s - CR_2 R_3 V s - R_2 V - R_3 V}{CLR_1 R_2 s^2 + CLR_2 R_3 s^2 + CR_1 R_2 R_3 s + LR_1 s + LR_2 s + LR_3 s + R_1 R_2 + R_1 R_3}
\end{aligned} \tag{2}$$

Numerical values can be substituted into the equations.

```
element_values = SymMNA.get_part_values(network_df)

# substitute numerical values
NE = NE_sym.subs(element_values)

# solve and display
U = solve(NE,X)

temp = ''
for i in U.keys():
```

```
temp += '${:s} = {:s}$<br>'.format(latex(i), latex(U[i]))
Markdown(temp)
```

$$\begin{aligned}
 v_1 &= \frac{474s + 216}{98s^2 + 273s + 108} \\
 v_2 &= \frac{-196s^2 - 72s}{98s^2 + 273s + 108} \\
 v_3 &= \frac{418s}{98s^2 + 273s + 108} \\
 I_V &= \frac{-28s^2 - 266s - 72}{98s^2 + 273s + 108} \\
 I_L &= \frac{-196s - 72}{98s^2 + 273s + 108}
 \end{aligned} \tag{3}$$

The circuit's node voltages and unknown currents can be solved for at individual frequencies. In the code cell below, a frequency of 1 Hz is used to evaluate the equations at a particular frequency.

```
freq = 1 # Hz
omega = 2*np.pi*freq

NE = NE_sym.subs(element_values)
NE_w1 = NE.subs({s:1j*omega})

U_w1 = solve(NE_w1,X)

table_header = ['unknown', 'mag', 'phase, deg']
table_row = []

for name, value in U_w1.items():
    table_row.append([str(name), float(abs(value)), float(arg(value)*180/np.pi)])

print(tabulate(table_row, headers=table_header, colalign =
('left', 'decimal', 'decimal'), tablefmt="simple", floatfmt=('.5s', '.6f', '.6f')))
```

See Table 1

Table 1: Calculation results.

unknown	mag	phase, deg
v_1	0.722388	-69.630838
v_2	1.875123	-158.828643
v_3	0.635373	-65.482650
I_V	0.475375	146.246200
I_L	0.298435	111.171357

The results obtained above agree with the solution generated by LTSpice.

3.2. Example 2

A transfer function is a fundamental mathematical representation used in circuit design to characterize the relationship between a circuit's output signal and its input signal. Specifically, it is defined as the ratio of the Laplace transform of the output (response) to the Laplace transform of the input (excitation), assuming all initial conditions are zero. This function, typically denoted as $H(s)$ where s is the complex frequency variable ($j\omega$), allows engineers to analyze and predict the circuit's behavior across different frequencies. By examining the poles (roots of the denominator) and zeros (roots of the numerator) of the transfer function, one can determine crucial circuit properties such as gain, stability, frequency response (e.g., whether it acts as a low-pass or high-pass filter), and transient behavior, making it an indispensable tool for designing and optimizing filters, amplifiers, and control systems.

This example walks through the Python code used to find the transfer function of the circuit shown in Figure 2.

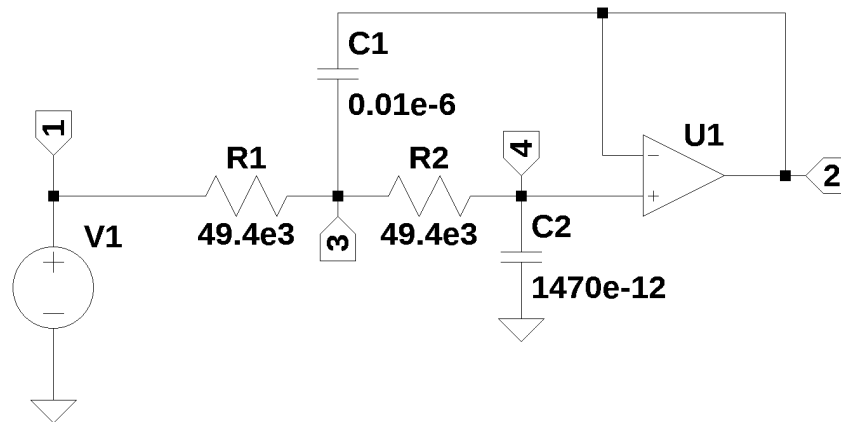


Figure 2: Schematic for example 2.

The circuit above is a 2nd order low pass filter from [4]. The schematic of the circuit is shown with each node annotated. The filter shown is being used to illustrate the application of Python to obtain the transfer function, $\frac{V_2}{V_1}$.

This circuit has two complex poles and by proper choice of R_1 , R_2 , C_1 and C_2 the complex poles can be positioned to realize Bessel, Butterworth or Chebyshev filter characteristics. The component values shown in the schematic produce a low pass filter with a cut off frequency of 1000 Hz and a 3 dB Chebyshev response.

The Python MNA code supports the Op Amp component type designated with the letter 'O' and is shown on the send line of the netlist

```
net_list = '''
R1 3 1 49.4e3
O1 2 4 2
V1 1 0 1
R2 4 3 49.4e3
C2 4 0 1470e-12
C1 2 3 0.01e-6
'''
```

```

# call the symbolic modified nodal analysis function to generate the MNA matrices.
report, network_df, i_unk_df, A, X, Z = SymMNA.smna(net_list)

# assemble and build the network equations.
# Put matrices into SymPy
X = Matrix(X)
Z = Matrix(Z)

NE_sym = Eq(A*X,Z)

# turn the free symbols into SymPy variables.
var(str(NE_sym.free_symbols).replace('{', '').replace('}', ''))

# construct a dictionary of element values.
element_values = SymMNA.get_part_values(network_df)

# display the equations
temp = ''
for i in range(shape(NE_sym.lhs)[0]):
    temp += '${:s} = {:s}$<br>'.format(latex(NE_sym.rhs[i]), latex(NE_sym.lhs[i]))
Markdown(temp)

```

$$\begin{aligned}
 0 &= I_{V_1} + \frac{v_1}{R_1} - \frac{v_3}{R_1} \\
 0 &= C_1 s v_2 - C_1 s v_3 + I_{O_1} \\
 0 &= -C_1 s v_2 + v_3 \left(C_1 s + \frac{1}{R_2} + \frac{1}{R_1} \right) - \frac{v_4}{R_2} - \frac{v_1}{R_1} \\
 0 &= v_4 \left(C_2 s + \frac{1}{R_2} \right) - \frac{v_3}{R_2} \\
 V_1 &= v_1 \\
 0 &= v_2 - v_4
 \end{aligned} \tag{4}$$

Use the SymPy function `solve` to determine the unknown node voltages and device currents.

```

U_sym = solve(NE_sym,X)

temp = ''
for i in U_sym.keys():
    temp += '${:s} = {:s}$<br>'.format(latex(i), latex(U_sym[i]))
Markdown(temp)

```

$$\begin{aligned}
v_1 &= V_1 \\
v_2 &= \frac{V_1}{C_1 C_2 R_1 R_2 s^2 + C_2 R_1 s + C_2 R_2 s + 1} \\
v_3 &= \frac{C_2 R_2 V_1 s + V_1}{C_1 C_2 R_1 R_2 s^2 + C_2 R_1 s + C_2 R_2 s + 1} \\
v_4 &= \frac{V_1}{C_1 C_2 R_1 R_2 s^2 + C_2 R_1 s + C_2 R_2 s + 1} \\
I_{V_1} &= \frac{-C_1 C_2 R_2 V_1 s^2 - C_2 V_1 s}{C_1 C_2 R_1 R_2 s^2 + C_2 R_1 s + C_2 R_2 s + 1} \\
I_{O_1} &= \frac{C_1 C_2 R_2 V_1 s^2}{C_1 C_2 R_1 R_2 s^2 + C_2 R_1 s + C_2 R_2 s + 1}
\end{aligned} \tag{5}$$

The symbolic transfer function, $H_{\text{sym}} = \frac{v_2}{v_1}$.

```
H_sym = (U_sym[v2]/U_sym[v1]).simplify().collect(s)
Markdown('$H(s) = \{s\}$'.format(latex(H_sym)))
```

$$H(s) = \frac{1}{C_1 C_2 R_1 R_2 s^2 + s(C_2 R_1 + C_2 R_2) + 1} \tag{6}$$

As shown above the denominator of $H(s)$ is a 2nd order polynomial in s . When using the standardized notation for damping ratio and natural frequency, the coefficients of the denominator polynomial can be equated to these quantities and expressed in terms of the circuit's component values. The standard notation is: $s^2 + 2\zeta\omega_n s + \omega_n^2$, where ζ is the damping ratio and ω_n is the natural frequency. The Quality Factor (Q) is a dimensionless quantity representing energy loss per cycle relative to the stored energy.

Getting the numerator and denominator of $H(s)$ by using the `fraction` function, which returns a pair with the expression's numerator and denominator.

```
H_sym_num, H_sym_denom = fraction(H_sym)
```

The denominator polynomial is:

```
Markdown('$D(s)=\{s\}$'.format(latex(H_sym_denom.collect(s))))
```

$$D(s) = C_1 C_2 R_1 R_2 s^2 + s(C_2 R_1 + C_2 R_2) + 1$$

The coefficients of each Laplace term can be equated to the variables b_2 , b_1 and b_0 in the expression:

$$b_2 s^2 + b_1 s + b_0$$

where b_2 , b_1 and b_0 are:

$$\begin{aligned}
b_2 &= C_1 C_2 R_1 R_2 \\
b_1 &= C_2 R_1 + C_2 R_2 \\
b_0 &= 1
\end{aligned} \tag{7}$$

Put the component values into the equations and Display the network equations with numerical component values.

Solve for voltages and currents and display the results.

```
NE = NE_sym.subs(element_values)
U = solve(NE,X)
temp = ''
for i in U.keys():
    temp += '${:s} = {:s}$<br>'.format(latex(i), latex(U[i]))
Markdown(temp)
```

$$\begin{aligned}
v_1 &= 1.0 \\
v_2 &= \frac{1.36591868959225 \cdot 10^{28}}{4.9 \cdot 10^{20} s^2 + 1.9838056680162 \cdot 10^{24} s + 1.36591868959225 \cdot 10^{28}} \\
v_3 &= \frac{9.91902834008098 \cdot 10^{23} s + 1.36591868959225 \cdot 10^{28}}{4.9 \cdot 10^{20} s^2 + 1.9838056680162 \cdot 10^{24} s + 1.36591868959225 \cdot 10^{28}} \\
v_4 &= \frac{1.36591868959225 \cdot 10^{28}}{4.9 \cdot 10^{20} s^2 + 1.9838056680162 \cdot 10^{24} s + 1.36591868959225 \cdot 10^{28}} \\
I_{V_1} &= \frac{-9.91902834008098 \cdot 10^{26} s^2 - 2.0079004737006 \cdot 10^{30} s}{4.9 \cdot 10^{31} s^2 + 1.9838056680162 \cdot 10^{35} s + 1.36591868959225 \cdot 10^{39}} \\
I_{O_1} &= \frac{9.91902834008098 \cdot 10^{15} s^2}{4.9 \cdot 10^{20} s^2 + 1.9838056680162 \cdot 10^{24} s + 1.36591868959225 \cdot 10^{28}}
\end{aligned} \tag{8}$$

Plot the frequency response of the transfer function

```
H = U[v2]/U[v1]
Markdown('${H(s)} = ${:s}$'.format(latex(H)))
```

$$H(s) = \frac{1.36591868959225 \cdot 10^{28}}{4.9 \cdot 10^{20} s^2 + 1.9838056680162 \cdot 10^{24} s + 1.36591868959225 \cdot 10^{28}} \tag{9}$$

```
num, denom = fraction(H) #returns numerator and denominator
# convert symbolic to numpy polynomial
a = np.array(Poly(num, s).all_coeffs(), dtype=float)
b = np.array(Poly(denom, s).all_coeffs(), dtype=float)
```

```
x = np.logspace(2, 4, 2000, endpoint=False)*2*np.pi
w, mag, phase = signal.bode((a, b), w=x) # returns: rad/s, mag in dB, phase in deg
```

The magnitude response and phase response together form the frequency response of a transfer function, $H(s)$, which is essential for understanding how a circuit processes different sinusoidal input signals. The frequency response is obtained by substituting $s = j\omega$ into the transfer function, yielding the complex function $H(j\omega)$. The magnitude response, $|H(j\omega)|$, represents the gain or attenuation of the circuit at a specific angular frequency ω . It shows how the amplitude of the output signal changes relative to the input signal's amplitude as the input frequency varies. The phase response, $\angle H(j\omega)$, represents the phase shift (or delay) introduced by the circuit between the output and input signals at that same frequency ω . These two components are typically plotted separately against frequency on a Bode plot and are crucial for applications like filter design, where specific gain and phase characteristics are required over a range of frequencies.

Plot the results.

```
fig, ax1 = plt.subplots()
ax1.set_ylabel('magnitude, dB')
ax1.set_xlabel('frequency, Hz')

plt.semilogx(w/(2*np.pi), mag, '-b') # Bode magnitude plot

ax1.tick_params(axis='y')
plt.grid()

# instantiate a second y-axes that shares the same x-axis
ax2 = ax1.twinx()
color = 'tab:blue'

plt.semilogx(w/(2*np.pi), phase, ':', color='tab:red') # Bode phase plot

ax2.set_ylabel('phase, deg', color=color)
ax2.tick_params(axis='y', labelcolor=color)

plt.title('Magnitude and phase response')
plt.savefig('eg2_mag&phase.png')
plt.show()
```

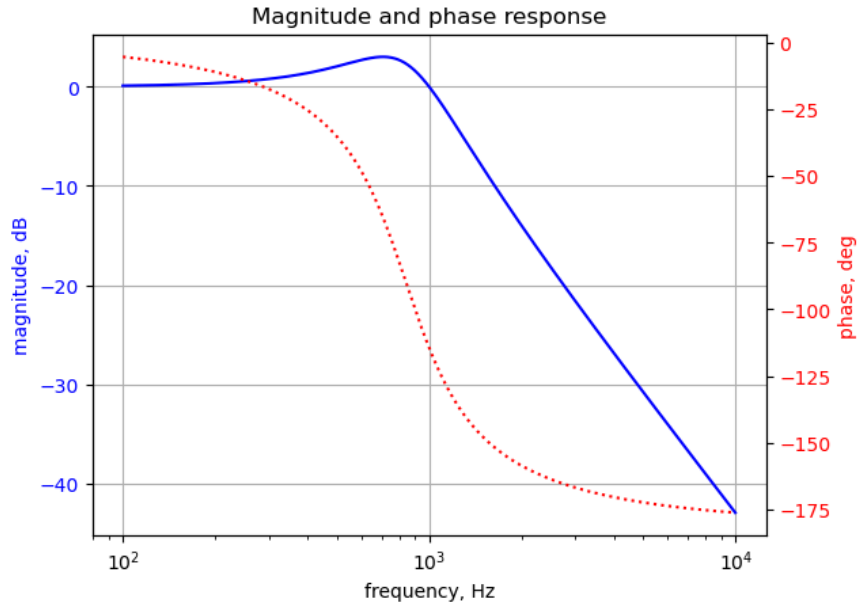


Figure 3: Mag and Phase for example 2.

The poles and zeros of the transfer function can easily be obtained with the following code:

The pole-zero plot, also known as the s -plane plot, is a graphical representation of a transfer function, $H(s)$, where the locations of its poles and zeros are marked on the complex s -plane. The zeros (roots of the numerator $N(s)$) are traditionally marked with a circle ($\circ$), and the poles (roots of the denominator $D(s)$) are marked with a cross ($\times$). The horizontal axis of the s -plane represents the real part (σ) which relates to the damping and transient response (decay or growth), while the vertical axis represents the imaginary part ($j\omega$) which relates to the oscillation frequency and sinusoidal steady-state response. This plot is a crucial visual aid because the relative positions of the poles and zeros immediately reveal key insights into the circuit's characteristics, such as its stability, natural frequencies, and how it will selectively amplify or attenuate signals at different frequencies, which is fundamental in filter design.

The proximity of a poles to the imaginary axis ($j\omega$ axis) in the pole-zero plot has a profound impact on the frequency response of the circuit. The closer a pole is to the $j\omega$ axis, the greater its influence on the magnitude of the transfer function $|H(j\omega)|$ for frequencies near the pole's imaginary part. Specifically, a pole close to the axis corresponds to a higher Q-factor (quality factor), resulting in a sharp peak or resonance in the magnitude response curve at the pole's natural frequency, ω_d (which is the imaginary part of the pole). This means the circuit will selectively amplify signals at that specific frequency. Conversely, a pole located farther to the left (more negative real part σ) corresponds to a highly damped system, resulting in a much broader and flatter frequency response peak, or potentially no noticeable peak at all, as the system's energy quickly dissipates.

The poles and zeros of the preamp transfer function are plotted.

```
sys = signal.TransferFunction(a,b)

sys_zeros = np.roots(sys.num)
sys_poles = np.roots(sys.den)
```

```
plt.plot(np.real(sys_zeros), np.imag(sys_zeros), 'ob', markerfacecolor='none')
plt.plot(np.real(sys_poles), np.imag(sys_poles), 'xr')
plt.legend(['Zeros', 'Poles'], loc=0)
plt.title('Pole / Zero Plot')
plt.xlabel('real part, \u03B1')
plt.ylabel('imaginary part, j\u03C9')
plt.xlim((-2500,0))
plt.ylim((-6000,6000))
plt.grid()
plt.savefig('eg2_pole&zero.png')
plt.show()
```

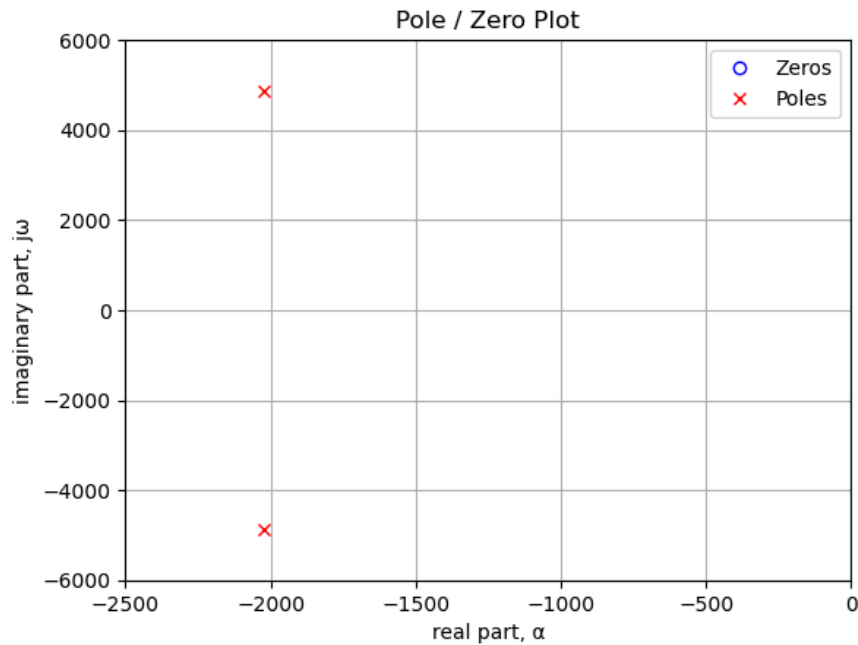


Figure 4: Pole and Zero plot for example 2.

Poles and zeros of the transfer function plotted on the complex plane. The units are in radian frequency.

The impulse response, $h(t)$, and the step response, $r(t)$, are two fundamental time-domain characteristics of a circuit defined by its transfer function, $H(s)$. The impulse response is the circuit's output when the input is an ideal Dirac delta function, $\delta(t)$, which is useful because $h(t)$ is the inverse Laplace transform of the transfer function itself: $h(t) = \mathcal{L}^{-1}\{H(s)\}$. It essentially reveals the natural, unforced behavior of the system. In contrast, the step response is the output when the input is a unit step function, $u(t)$ (a signal that jumps from zero to one at $t = 0$), and it is related to the impulse response by integration: $r(t) = \int_0^t h(\tau) d\tau$. Analyzing the step response is especially practical in circuit testing as it allows engineers to easily assess key performance metrics like rise time, settling time, overshoot, and steady-state error, all of which are crucial for evaluating the speed and quality of a circuit's transient behavior.

Rise time and settling time are key metrics derived from the step response that quantify the speed and convergence of a circuit's transient behavior.

- Rise Time (t_r): This is the time required for the step response to go from 10% of its final value to 90% of its final value. A shorter rise time indicates a faster circuit response. It is a measure of how quickly the system can react to a sudden change in input.
- Settling Time (t_s): This is the time required for the step response to reach and stay within a specified tolerance band around its final, steady-state value. The tolerance band is typically 2% or 5% of the final value. A common definition uses the 2% band. A shorter settling time indicates that the oscillations or transient effects damp out quickly, meaning the circuit stabilizes faster and is a measure of the circuit's overall speed and damping.

Both measurements are essential for evaluating system performance, where rise time reflects the initial speed, and settling time reflects the overall stability and convergence of the output.

Use the SciPy functions `impulse` and `step` to plot the impulse and step response of the system.

```
plt.subplots(1,2,figsize=(15, 5))

# using subplot function and creating
# plot one
plt.subplot(1, 2, 1)

# impulse response
t, y = signal.impulse(sys,N=500)
plt.plot(t/1e-3, y)
plt.title('Impulse response')
plt.ylabel('volts')
plt.xlabel('time, msec')
plt.grid()

# using subplot function and creating plot two
plt.subplot(1, 2, 2)

t, y = signal.step(sys,N=500)
plt.plot(t/1e-3, y)
plt.title('Step response')
plt.ylabel('volts')
plt.xlabel('time, msec')
plt.grid()

# show plot
plt.savefig('eg2_impulse&step.png')
plt.show()
```

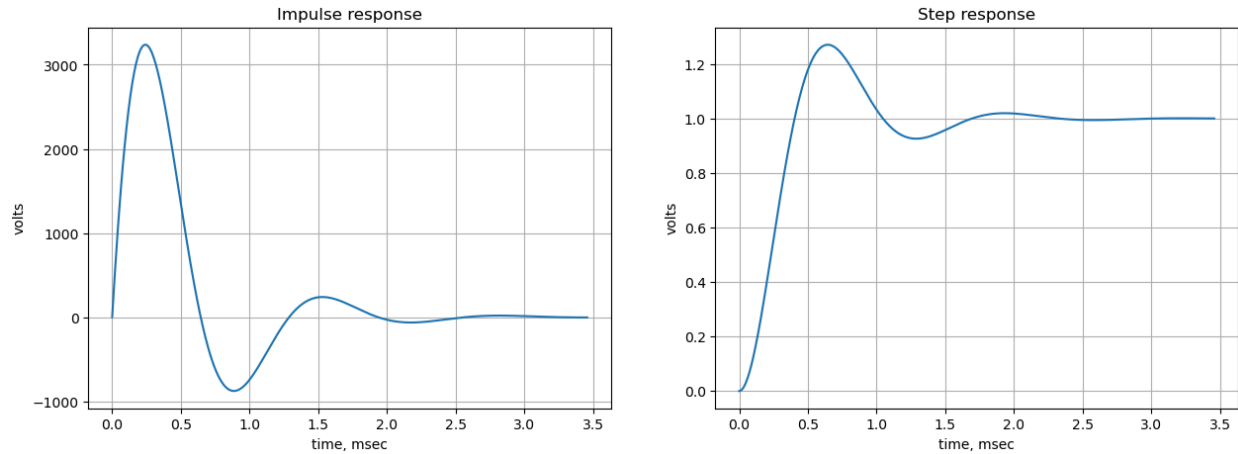


Figure 5: Imp and Step for example 2.

3.3. Example 3 - Transient Analysis

Python code is finished, see example 3 in *Circuit_analysis_w_python.ipynb* and finish write up and code for display of equations.

See [5]

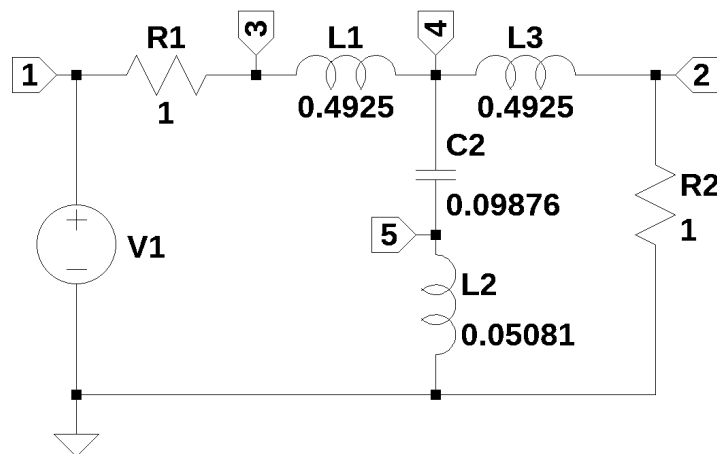


Figure 6: Schematic for example 3.

```
net_list = '''
V1 1 0 1
R1 3 1 1
R2 2 0 1
L1 3 4 0.4925
L2 5 0 0.05081
C2 4 5 0.09876
L3 4 2 0.4925
'''

report, network_df, df2, A, X, Z = SymMNA.smna(net_list)

# Put matrices into SymPy
```

```

X = Matrix(X)
Z = Matrix(Z)

NE_sym = Eq(A*X,Z)

# The symbols generated by the Python code are extracted by the SymPy function free_symbols
# and then declared as SymPy variables.
var(str(NE_sym.free_symbols).replace('{', '').replace('}', ''))

# built a dictionary of element values.
element_values = SymMNA.get_part_values(network_df)

# generate markdown text to display the network equations.
temp = ''
for i in range(len(X)):
    temp += '${:s}${<br>'.format(latex(Eq((A*X)[i:i+1][0],Z[i])))

Markdown(temp)

```

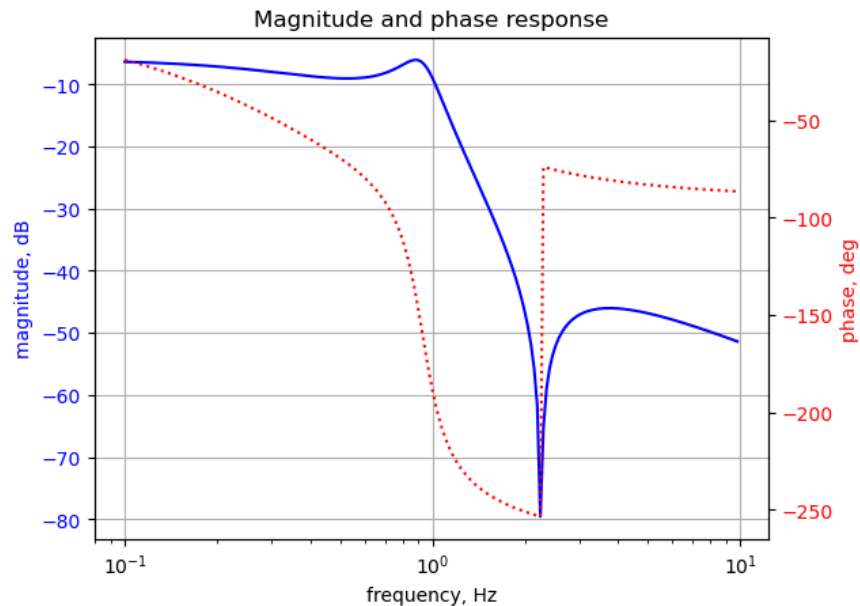


Figure 7: Mag and Phase for example 3.

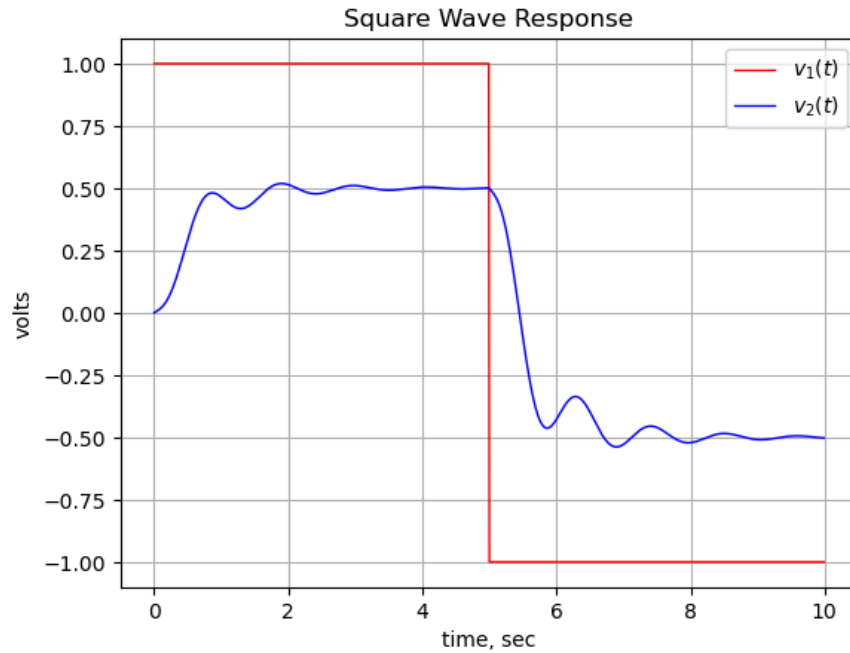


Figure 8: Squarewave for example 3.

3.4. Example 4 - Mutual Inductance

Use Alternate Example 4 - Mutual Inductance in *Circuit_analysis_w_python.ipynb* and finish write up and code for display of equations.

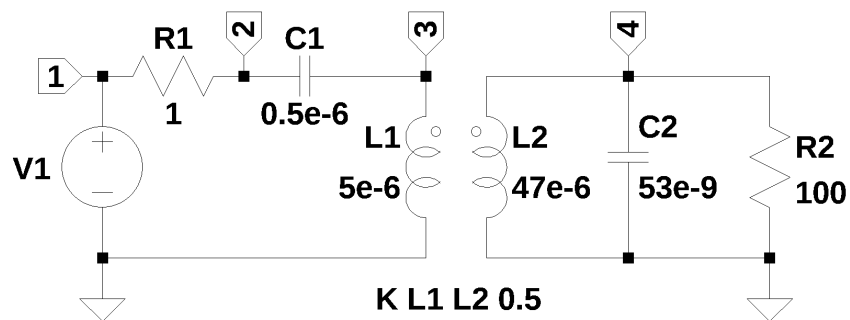


Figure 9: Schematic for example 4.

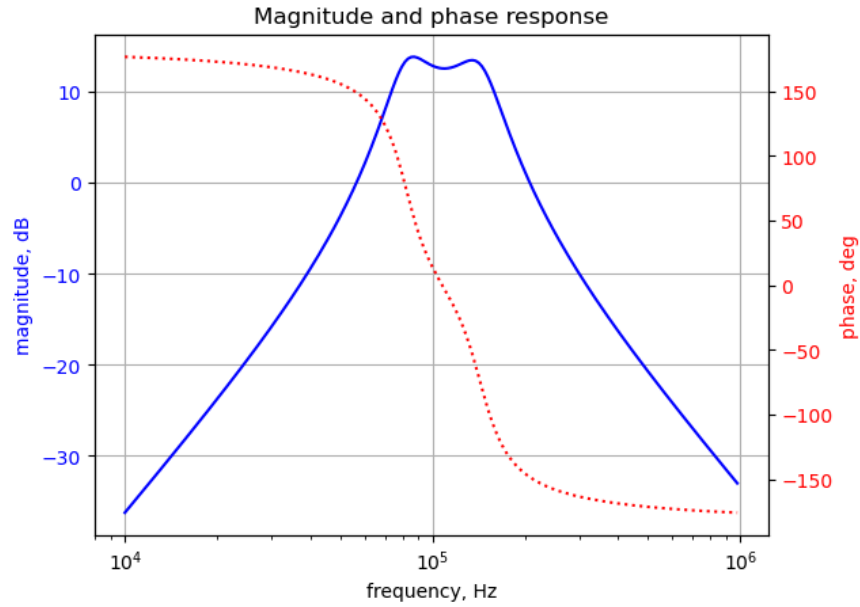


Figure 10: Mag and Phase for example 4.

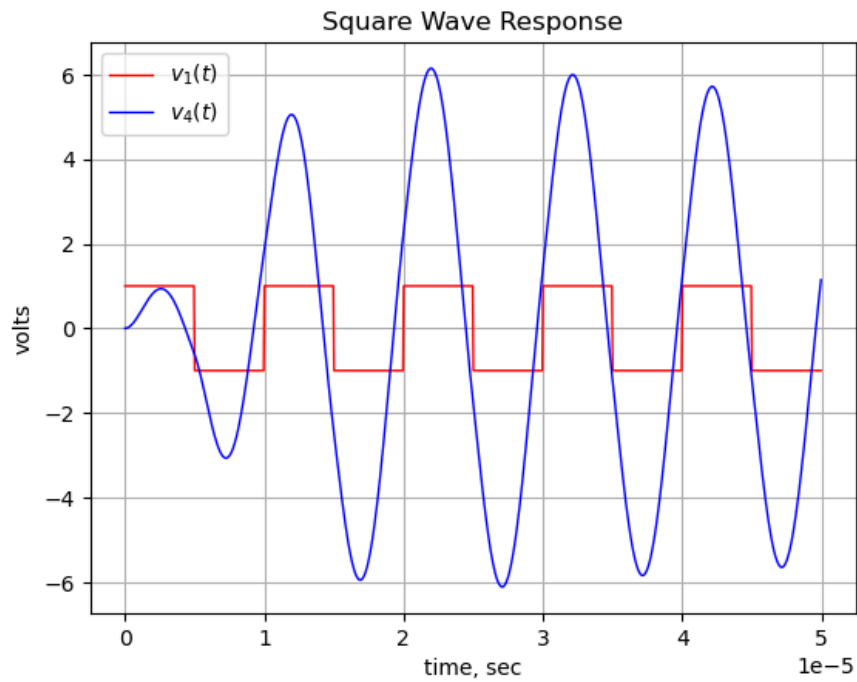


Figure 11: Squarewave for example 4.

3.5. Example 5 - Initial Conditions

Finish this section.

See [6] following answer from Davide_sd.

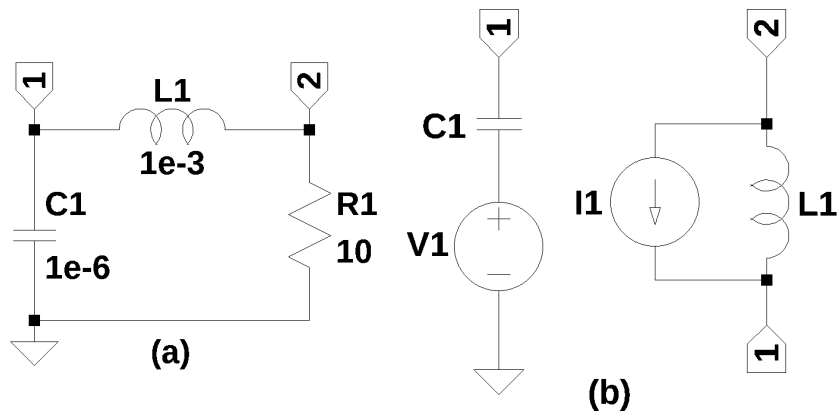


Figure 12: Schematic for example 4.

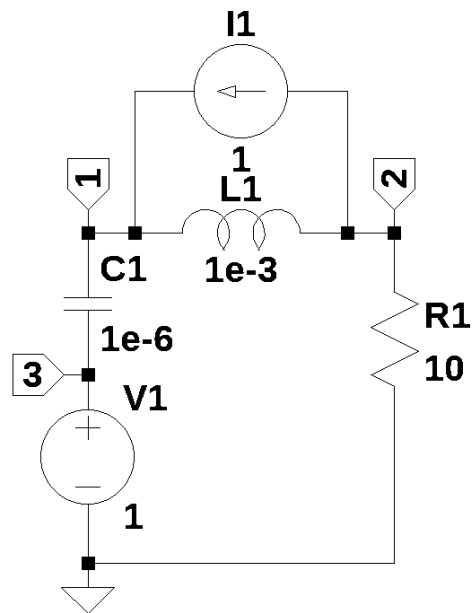


Figure 13: Mag and Phase for example 4.

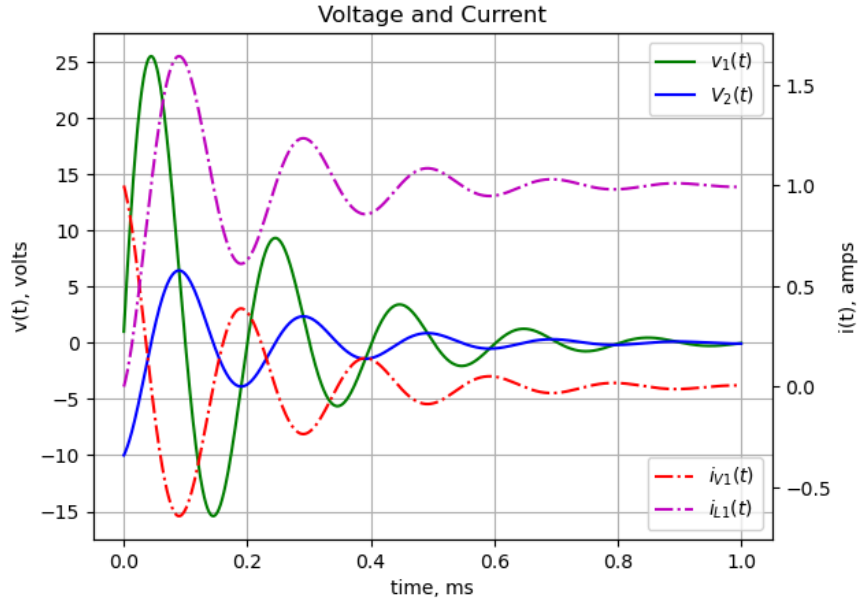


Figure 14: Squarewave for example 4.

3.6. Example 6 - Controlled Sources

The final example is the circuit shown in Figure 15. This circuit has a variety of dependent and independent sources. There are 13 branches in the circuit and eight nodes in the circuit. The circuit has six resistors, one capacitor and one inductor along with two independent voltage sources and one independent current source.

V_1 is an independent AC voltage source, I_1 is an independent DC current source, G is a voltage controlled current source and H is a current controlled voltage source, with the controlling current determined by the current in V_2 , which has a voltage value of zero.

The circuit analysis involves a two-step process based on the superposition theorem.

Step 1: The DC operating point of the circuit is calculated by setting:

- DC voltage and current sources are set to their specified values.
- All independent AC sources are set to zero.
- The Laplace variable, $s = j\omega$ is zero since $\omega = 0$.

Step 2: The frequency-Domain AC Analysis is calculated by setting:

- All independent DC sources are set to zero. They set the operating point in Step 1, but contribute zero signal amplitude in Step 2.
- The AC source generates a unit sinusoidal signal at the frequencies specified when ω is assigned a value.

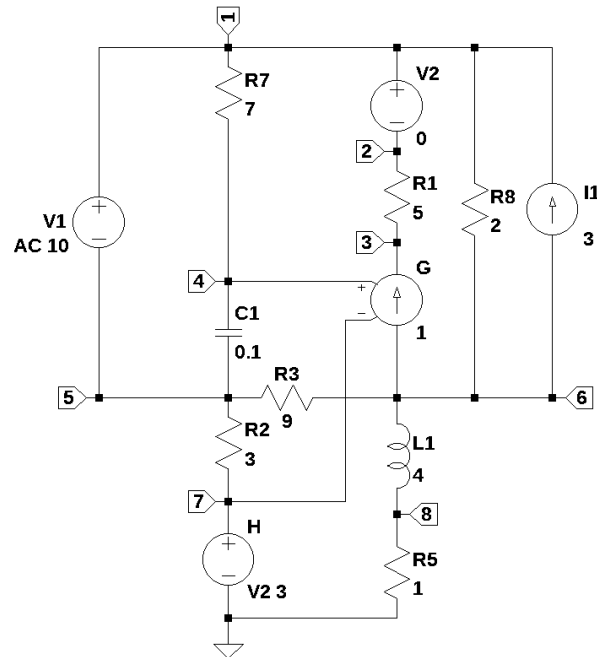


Figure 15: This schematic contains a variety of sources.

```
net_list = '''
* Controlled_sources_example_v0.asc
I1 6 1 3
G 6 4 3 7 1
H 7 0 V2 3
V1 1 5 10
C1 3 5 0.1
L1 6 8 4
R1 2 4 5
R2 5 7 3
R3 6 5 9
V2 1 2 0
R5 8 0 1
R7 1 3 7
R8 1 6 2
'''

# generate the MNA matrices.
report, network_df, i_unk_df, A, X, Z = SymMNA.smna(net_list)

# Put matrices into SymPy
X = Matrix(X)
Z = Matrix(Z)

# assemble the system of equations
NE_sym = Eq(A*X, Z)

# turn the free symbols into SymPy variables.
var(str(NE_sym.free_symbols).replace('{', '').replace('}', ''))

# display the network equations
```

```
temp = ''
for i in range(len(X)):
    temp += '${:s}$<br>'.format(latex(Eq((A*X)[i:i+1][0],Z[i])))
Markdown(temp)
```

$$\begin{aligned}
I_1 &= I_{V_1} + I_{V_2} + v_1 \left(\frac{1}{R_8} + \frac{1}{R_7} \right) - \frac{v_6}{R_8} - \frac{v_3}{R_7} \\
0 &= -I_{V_2} + \frac{v_2}{R_1} - \frac{v_4}{R_1} \\
0 &= -C_1 s v_5 + v_3 \left(C_1 s + \frac{1}{R_7} \right) - \frac{v_1}{R_7} \\
0 &= -g v_3 + g v_7 - \frac{v_2}{R_1} + \frac{v_4}{R_1} \\
0 &= -C_1 s v_3 - I_{V_1} + v_5 \left(C_1 s + \frac{1}{R_3} + \frac{1}{R_2} \right) - \frac{v_6}{R_3} - \frac{v_7}{R_2} \\
-I_1 &= I_{L_1} + g v_3 - g v_7 + v_6 \left(\frac{1}{R_8} + \frac{1}{R_3} \right) - \frac{v_1}{R_8} - \frac{v_5}{R_3} \\
0 &= I_H - \frac{v_5}{R_2} + \frac{v_7}{R_2} \\
0 &= -I_{L_1} + \frac{v_8}{R_5} \\
V_1 &= v_1 - v_5 \\
V_2 &= v_1 - v_2 \\
0 &= -I_{V_2} h + v_7 \\
0 &= -I_{L_1} L_1 s + v_6 - v_8
\end{aligned} \tag{10}$$

comment, see Equation 10 and

comments ...

more comments ... See Equation 11

$$\begin{aligned}
&C_1 L_1 R_7 V_1 s^2 (R_3 + R_8) + s(-C_1 I_1 R_2 R_3 R_7 R_8 g h + C_1 I_1 R_2 R_3 R_7 R_8 - \\
&C_1 R_2 R_3 R_7 R_8 V_1 g - C_1 R_2 R_7 R_8 V_1 g h + C_1 R_2 R_7 R_8 V_1 + C_1 R_3 R_5 R_7 V_1 + \\
&C_1 R_3 R_7 R_8 V_1 + C_1 R_5 R_7 R_8 V_1 - L_1 R_3 V_1 g h + L_1 R_3 V_1 - L_1 R_8 V_1 g h + \\
&L_1 R_8 V_1) - (g h - 1)(I_1 R_2 R_3 R_8 + R_2 R_8 V_1 + R_3 R_5 V_1 + R_3 R_8 V_1 + R_5 R_8 V_1) \\
v_1(s) &= \frac{C_1 L_1 R_7 s^2 (R_3 + R_8) - R_2 R_3 R_8 g - R_2 R_3 g h + R_2 R_3 - R_2 R_8 g h + R_2 R_8 +}{C_1 L_1 R_7 s^2 (R_3 + R_8) - R_2 R_3 R_8 g - R_2 R_3 g h + R_2 R_3 - R_2 R_8 g h + R_2 R_8 +} \tag{11} \\
&R_3 R_5 + R_3 R_8 + R_5 R_8 + s(-C_1 R_2 R_3 R_7 R_8 g - C_1 R_2 R_3 R_7 g h + C_1 R_2 R_3 R_7 - \\
&C_1 R_2 R_7 R_8 g h + C_1 R_2 R_7 R_8 + C_1 R_3 R_5 R_7 + C_1 R_3 R_7 R_8 + C_1 R_5 R_7 R_8 + \\
&L_1 R_3 + L_1 R_8)
\end{aligned}$$

See Table 2

Table 2: Probe results for design A

unknown	Op Point	Units
v_1	3.560440	volts
v_2	3.560440	volts
v_3	3.560440	volts
v_4	-5.340659	volts
v_5	3.560440	volts
v_6	0.593407	volts
v_7	5.340659	volts
v_8	0.593407	volts
I_{V1}	-0.263736	amps
I_{V2}	1.780220	amps
I_H	-0.593407	amps
I_{L1}	0.593407	amps

See Table 3

Table 3: Probe results for design A

unknown	mag	phase, deg	Units
v_1	12.327333	39.304107	volts
v_2	12.327333	39.304107	volts
v_3	6.981493	26.458441	volts
v_4	6.086848	179.697882	volts
v_5	7.822187	93.379955	volts
v_6	14.192228	58.245503	volts
v_7	10.472239	26.458441	volts
v_8	3.442121	-17.718253	volts
I_{V1}	4.630794	-178.801918	amps
I_{V2}	3.490746	26.458441	amps
I_H	3.442121	162.281747	amps
I_{L1}	3.442121	-17.718253	amps

Table 4: AC analysis results

unknown	Op Point	Units
v_1	12.327333	39.30410677345189
v_2	12.327333	39.30410677345189
v_3	6.981493	26.458440999941015
v_4	6.086848	179.6978824010276
v_5	7.822187	93.37995495492912
v_6	14.192228	58.24550312351385
v_7	10.472239	26.458440999941015
v_8	3.442121	-17.718253408559672
I_{V1}	4.630794	-178.80191783666675
I_{V2}	3.490746	26.458440999941015
I_H	3.442121	162.28174659144034
I_{L1}	3.442121	-17.718253408559672

4. Discussion and Conclusion

This paper serves as a practical procedure for performing symbolic circuit analysis using the Python programming language and its associated scientific libraries, such as SymPy and NumPy. The primary goal is to provide a systematic and computationally efficient procedure for analyzing linear electric circuits, utilizing JupyterLab Notebooks as adaptable templates. MNA is an essential extension of standard nodal analysis that allows all types of circuit elements (including voltage sources, inductors, and dependent sources) to be incorporated into a single, unified matrix formulation.

Key Concepts:

- **Symbolic Analysis:** This technique calculates a circuit's behavior (e.g., gain, impedance) with components and frequency represented by symbols instead of numerical values. This provides qualitative insight into the circuit's operation, aids in design optimization, and yields exact analytical solutions that numerical simulators (like SPICE) cannot.
- **Modified Nodal Analysis (MNA):** The procedure leverages MNA to formulate the circuit problem as a system of simultaneous linear equations, $G \cdot V = I$, which is then solved using Linear Algebra techniques (matrix inversion) implemented in Python.
- **Advanced Concepts (The s-Domain):** The procedure utilizes Laplace Transformed Circuit Models, which convert time-domain differential equations into simpler algebraic equations in the s-domain (complex frequency domain). This is crucial for analyzing circuits with energy storage elements (capacitors and inductors) and determining their transient and transfer functions.

5. Python Code Availability

The Python code and computation results presented in this paper are contained in a JupyterLab notebook which is hosted on GitHub (https://github.com/Tiburonboy/EE_jupyter_notebooks/tree/main/Circuit_analysis_with_python).

References

- [1] C. Ho, A. Ruehli, and P. Brennan, "The modified nodal approach to network analysis," *IEEE Transactions on Circuits and Systems*, vol. 22, no. 6, pp. 504–509, 1975, doi: 10.1109/TCS.1975.1084079.
- [2] Tiburonboy, "Circuit Analysis with Python." Accessed: Dec. 07, 2025. [Online]. Available: https://github.com/Tiburonboy/EE_jupyter_notebooks/blob/main/Circuit_analysis_with_python/Circuit_analysis_w_python.ipynb
- [3] Tiburonboy, "Symbolic Modified Nodal Analysis using Python." Accessed: Aug. 13, 2025. [Online]. Available: <https://tiburonboy.github.io/Symbolic-Modified-Nodal-Analysis-using-Python/>
- [4] D. Stout, *Handbook of Operational Amplifier Circuit Design*. McGraw-Hill, 1976.
- [5] Marki Microwave, "LC Filter Design Tool." Accessed: Dec. 10, 2025. [Online]. Available: <https://markimicrowave.com/technical-resources/tools/lc-filter-design-tool/>
- [6] zorka5, "Calculating inverse laplace transform using python or matlab." Accessed: Dec. 10, 2025. [Online]. Available: <https://stackoverflow.com/questions/77876224/calculating-inverse-laplace-transform-using-python-or-matlab>